

Python - Copy Lists

Copying a List in Python

Copying a list in Python refers to creating a new list that contains the same elements as the original list. There are different methods for copying a list, including, using slice notation, the `list()` function, and using the `copy()` method.

Each method behaves differently in terms of whether it creates a shallow copy or a deep copy. Let us discuss about all of these deeply in this tutorial.

Shallow Copy on a Python List

A shallow copy in Python creates a new object, but instead of copying the elements recursively, it copies only the references to the original elements. This means that the new object is a separate entity from the original one, but if the elements themselves are mutable, changes made to those elements in the new object will affect the original object as well.

Example of Shallow Copy

Let us illustrate this with the following example –

```
import copy
# Original list
original_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
# Creating a shallow copy
shallow_copied_list = copy.copy(original_list)
# Modifying an element in the shallow copied list
shallow_copied_list[0][0] = 100
# Printing both lists
```

```
print("Original List:", original_list)
print("Shallow Copied List:", shallow_copied_list)
```

As you can see, even though we only modified the first element of the first sublist in the shallow copied list, the same change is reflected in the original list as well.

This is because a shallow copy only creates new references to the original objects, rather than creating copies of the objects themselves –

```
Original List: [[100, 2, 3], [4, 5, 6], [7, 8, 9]]
Shallow Copied List: [[100, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Deep Copy on a Python List

A deep copy in Python creates a completely new object and recursively copies all the objects referenced by the original object. This means that even nested objects within the original object are duplicated, resulting in a fully independent copy where changes made to the copied object do not affect the original object, and vice versa.

Example of Deep Copy

Let us illustrate this with the following example –



```
import copy
# Original list
original_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
# Creating a deep copy
deep_copied_list = copy.deepcopy(original_list)
# Modifying an element in the deep copied list
deep_copied_list[0][0] = 100
# Printing both lists
print("Original List:", original_list)
print("Deep Copied List:", deep_copied_list)
```

As you can see, when we modify the first element of the first sublist in the deep copied list, it does not affect the original list.

This is because a deep copy creates a new object and recursively copies all the nested objects, ensuring that the copied object is fully independent from the original one –

```
Original List: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
Deep Copied List: [[100, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Copying List Using Slice Notation

Slice notation in Python allows you to create a subsequence of elements from a sequence (like a list, tuple, or string) by specifying a start index, an end index, and an optional step size. The syntax for slice notation is as follows –

```
[start:end:step]
```

Where, **start** is the index where the slice starts, **end** is the index where the slice ends (exclusive), and **step** is the step size between elements.

We can copy a list using slice notation by specifying the entire range of indices of the original list. This effectively creates a new list with the same elements as the original list.

Any modifications made to the copied list will not affect the original list, and vice versa, because they are separate objects in memory.

Example

In this example, we are creating a slice of the "original_list", effectively copying all its elements into a new list "copied_list" –

```
# Original list  
original_list = [1, 2, 3, 4, 5]  
# Copying the list using slice notation  
copied_list = original_list[1:4]  
# Modifying the copied list  
copied_list[0] = 100  
# Printing both lists  
print("Original List:", original_list)  
print("Copied List:", copied_list)
```

We get the result as shown below –

Original List: [1, 2, 3, 4, 5]

Copied List: [100, 3, 4]

Copying List Using the list() Function

The list() function in Python is a built-in function used to create a new list object. It can accept an iterable (like another list, tuple, set, etc.) as an argument and create a new list containing the elements of that iterable. If no argument is provided, an empty list is created.

We can copy a list using the list() function by passing the original list as an argument. This will create a new list object containing the same elements as the original list.

Example

In the example below, we are creating a new list object "copied_list" containing the same elements as "original_list" using the list() function –

```
# Original list
original_list = [1, 2, 3, 4, 5]
# Copying the list using the list() constructor
copied_list = list(original_list)
# Printing both lists
print("Original List:", original_list)
print("Copied List:", copied_list)
```

Following is the output of the above code –

Original List: [1, 2, 3, 4, 5]

Copied List: [1, 2, 3, 4, 5]

Copying List Using the copy() Function

In Python, the copy() function is used to create a shallow copy of a list or other mutable objects. This function is part of the copy module in Python's standard library.

We can copy a list using the copy() function by invoking it on the original list. This creates a new list object that contains the same elements as the original list.

Example

In the following example, we are using the `copy()` function to create a new list object "copied_list" containing the same elements as "original_list" –

```
import copy
original_list = [1, 2, 3, 4, 5]
# Copying the list using the copy() function
copied_list = copy.copy(original_list)
print("Copied List:", copied_list)
```

Output of the above code is as shown below –

```
Copied List: [1, 2, 3, 4, 5]
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial
- Amazon Web Services Tutorial
- Microsoft Azure Tutorial

[Git Tutorial](#)
[Ethical Hacking Tutorial](#)
[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)
[Java & Spring Boot Advanced Certification](#)
[Data Science Advanced Certification](#)
[Cloud Computing And DevOps](#)
[Advanced Certification In Business Analytics](#)
[Artificial Intelligence And Machine Learning](#)
[DevOps Certification](#)
[Game Development Certification](#)
[Front-End Developer Certification](#)
[AWS Certification Training](#)
[Python Programming Certification](#)

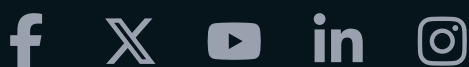


Chapters ▾

Categories ≡

[Online Java Compiler](#)
[Online Python Compiler](#)
[Online Go Compiler](#)
[Online C Compiler](#)
[Online C++ Compiler](#)
[Online C# Compiler](#)
[Online PHP Compiler](#)
[Online MATLAB Compiler](#)
[Online Bash Terminal](#)
[Online SQL Compiler](#)
[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.